



INTÉRPRETE SMART-HOME Y TRADUCTOR A HTML

INTEGRANTES

- *Balbuena Yusva, Mario Juan Salvador*
- *Baruzzo Ravanedo, Jorge Carlos Omar.*
- *Durand Bruno, Mariano.*
- *Escobar, Ramiro Gonzalo.*
- *Perez, Lucas Rodrigo*

ASIGNATURA

Sintaxis y Semántica de los Lenguajes.

CARRERA

Ingeniería en Sistemas de Información

FACULTAD

Universidad Tecnológica Nacional, Regional Resistencia.

CURSO

- K2.1
- K2.2

AÑO

- 2026

Índice

Introducción.....	3
Integrantes.....	3
Matriz de habilidades.....	3
Identidad.....	4
Alianza.....	5
Gramática.....	5
Análisis Léxico.....	11
Definición de Componente Léxicos.....	11
Estructura de datos del token.....	12
Mecanismo de reconocimiento y orden de precedencia.....	12
Resolución de contexto.....	13
Gestión de errores.....	13
Protocolo de recuperación ante caracteres inválidos.....	13
Sistema de advertencias semánticas tempranas.....	14
MÉTODO DE FUNCIONAMIENTO.....	14
Análisis Sintáctico.....	14

Introducción

El presente trabajo práctico integrador se desarrolla en el marco de la asignatura Sintaxis y Semántica de Lenguajes. El núcleo de este trabajo radica en la construcción de un parser. El proceso se aborda desde una perspectiva de resolución de problemas, descomponiendo el objetivo en etapas modulares e interdependientes que garantizan la integridad de la información procesada.

Conformación del grupo

Integrantes

Balbuena Yusva, Mario J. S.

Baruzzo Ravanedo, Jorge Carlos Omar.

Durand Bruno, Mariano.

Escobar, Ramiro Gonzalo.

Perez, Lucas Rodrigo

Matriz de habilidades

Integrantes	Inglés	Edición de video	Habilidades de Programación	Búsqueda de información	Documentación	Idear soluciones
Balbuena Yusva Mario Juan Salvador	O	★	★	★	O	O
Baruzzo Ravanedo, Jorge Carlos Omar	★	O	O	★	★	O
Durand Bruno Mariano	★	O	O	★	O	★
Escobar	O	O	★	★	★	O

Gonzalo						
Perez, Lucas Rodrigo	O	★	O	O	O	O

Identidad

Nombre: Token Hunters.

Logo:



Alianza

Como grupo, acordamos organizarnos de manera conjunta, responsable y constante para el desarrollo del trabajo propuesto por la cátedra con el objetivo principal de lograr un proyecto funcional y bien documentado, distribuyendo las tareas de forma equitativa y aprovechando las fortalezas individuales de cada integrante. Asegurando que los integrantes puedan desenvolverse con fluidez en áreas clave como la programación, la documentación y la planificación del proyecto.

La comunicación será de manera constante haciendo uso de herramientas como Discord y WhatsApp, para coordinar tareas, resolver dudas y hacer seguimiento del trabajo. Asimismo, emplearemos GitHub con el fin de facilitar el control de versiones del proyecto y mantener actualizado el mismo.

Se realizaremos reuniones virtuales al menos dos veces por semana, buscando un horario en que todo los integrantes o la gran mayoría puedan asistir. También, cada dos semanas, llevaremos a cabo una reunión presencial con el objetivo de intercambiar ideas, aclarar dudas y proponer nuevas soluciones.

Todos los integrantes nos comprometemos a trabajar de manera continua durante el tiempo establecido, evitando dejar tareas para último momento, con el fin de mantener una buena calidad del trabajo. Además, fomentaremos una actitud respetuosa, participativa y colaborativa, promoviendo el trabajo en equipo y el aprendizaje conjunto para cumplir de forma efectiva con los objetivos propuestos.

Gramática

Símbolos no terminales

No terminales = {

PROGRAMA, LISTA_INST, INSTRUCCION, BLOQUE,

WHEN_STMT, EVERY_STMT, IF_STMT,

CONDICION, COND_SIMPLE, COND_COMPUESTA,

EXPRESION, OPERADOR1, OPERADOR2, OPERADOR3,

ACCIONES, ACCION,

ASIGNACION,

IDENTIFICADOR, SENSOR, FOCO, AIRE, PERSIANA, CERRADURA, RELOJ,

ALTAVOZ, ALARMA,

TIEMPO, NUMERO, BOOLEANO1, BOOLEANO2, TEXTO, HORA, FECHA,

ESTADO1, ESTADO2, BRILLO1, BRILLO2, COLOR1, COLOR2, COLORES,

MODO1, MODO2, DISCRETO, TEMP_OBJ1, TEMP_OBJ2, TEMP_ACT,

POSICION1, POSICION2, VOLUMEN1, VOLUMEN2, MUTE1, MUTE2, MENSAJE1, MENSAJE2, EMAIL1, EMAIL2, ACTIVADA1, ACTIVADA2, TEMP, PORC, LUX }

Símbolos terminales

Terminales = {

WHEN, IF, THEN, ELSE, DO, END, EVERY,

AND, OR, NOT,

==, !=, >, <, >=, <=,

=, .,

ON, OFF, TRUE, FALSE,

NUM, °C, lux, %,

sensor_temp, sensor_luz, sensor_humo, sensor_movimiento, sensor_humedad,

foco_, aire_, persiana_cerradura_, reloj_, altavoz_, alarma_,

estado, brillo, color, modo, frío, calor, vent, temp_obj, temp_act, posicion, volumen,

notif_email, email, mensaje, mute, activada, hora:minuto:segundo, día:mes:año }

SIGMA

$\Sigma \rightarrow \text{PROGRAMA}$

PROGRAMA

PROGRAMA $\rightarrow$ LISTA_INST

LISTA_INST $\rightarrow$ INSTRUCCION

LISTA_INST $\rightarrow$ INSTRUCCION LISTA_INST

INSTRUCCIONES

INSTRUCCION $\rightarrow$ WHEN_STMT

INSTRUCCION $\rightarrow$ EVERY_STMT

INSTRUCCION $\rightarrow$ IF_STMT

INSTRUCCION $\rightarrow$ ASIGNACION

WHEN

WHEN_STMT $\rightarrow$ WHEN CONDICION DO ACCIONES END

EVERY

EVERY_STMT → EVERY TIEMPO DO ACCIONES END

IF

IF_STMT → IF CONDICION THEN ACCIONES ELSE ACCIONES

IF_STMT → IF CONDICION THEN ACCIONES

ACCIONES

ACCIONES → ACCION

ACCIONES → ACCION ACCIONES

ACCION

ACCION → ASIGNACION

ACCION → IF_STMT

ASIGNACION

**ASIGNACION → foco_.ESTADO2 | foco_.BRILLO2 | foco_.COLOR2 |
aire_.ESTADO2 | aire_.MOD02 | aire_.TEMP_OBJ2 | persiana_.POSICION2 |
cerradura_.ESTADO2 | altavoz_.VOLUMEN2 | altavoz_.MUTE2 |
altavoz_.MENSAJE2 | altavoz_.EMAIL2 | alarma_.ESTADO2 |
alarma_.ACTIVADA2**

CONDICION SIMPLE

COND_SIMPLE → EXPRESION

CONDICION COMPUESTA

COND_COMPUESTA → COND_SIMPLE AND COND_SIMPLE

COND_COMPUESTA → COND_SIMPLE OR COND_SIMPLE

COND_COMPUESTA → NOT COND_SIMPLE

CONDICIONES

CONDICION → COND_SIMPLE

CONDICION → COND_COMPUESTA

EXPRESION

EXPRESION → IDENTIFICADOR

IDENTIFICADOR

**IDENTIFICADOR → SENSOR | FOCO | AIRE | PERSIANA | CERRADURA |
RELOJ | ALTAVOZ | ALARMA**

**SENSOR → sensor_temp OP_IGUAL TEMP | sensor_temp OP_COMP TEMP |
sensor_humedad OP_IGUAL PORC | sensor_humedad OP_COMP PORC |
sensor_luz OP_IGUAL LUX | sensor_luz OP_COMP LUX | sensor_movimiento
OP_IGUAL BOOLEANO1 | sensor_humo OP_IGUAL BOOLEANO1**

FOCO → foco_.ESTADO1 | foco_.BRILLO1 | foco_.COLOR1

ESTADO1 → estado OP_IGUAL BOOLEANO2

ESTADO2 → estado OP_ASSIGN BOOLEANO3

BRILLO1 → brillo OP_IGUAL PORC

BRILLO2 → brillo OP_ASSIGN PORC

COLOR1 → color OP_IGUAL COLORES

COLOR2 → color OP_ASSIGN COLORES

COLORES → blanco | azul | rojo

AIRE → aire_.ESTADO1 | aire_.MODO1 | aire_.TEMP_OBJ1 | aire_.TEMP_ACT

MODO1 → modo OP_IGUAL DISCRETO

MODO2 → modo OP_ASSIGN DISCRETO

DISCRETO → frio | calor | vent

TEMP_OBJ1 → temp_obj OP_IGUAL TEMP

TEMP_OBJ2 → temp_obj OP_ASSIGN TEMP

TEMP_ACT → temp_act OP_IGUAL TEMP | temp_act OP_COMP TEMP

PERSIANA → persiana_.POSICION1

POSICION1 → posición OP_IGUAL PORC | posición OP_COMP PORC

POSICION2 → posición OP_ASSIGN PORC

CERRADURA → cerradura_.ESTADO1

**RELOJ → reloj_.HORA OP_IGUAL HORA | reloj_.FECHA OP_IGUAL FECHA |
reloj_.HORA OP_COMP HORA | reloj_.FECHA OP_COMP FECHA**

**ALTAVOZ → altavoz_.VOLUMEN1 | altavoz_.MUTE1 | altavoz_.MENSAJE1 |
altavoz_.EMAIL1**

VOLUMEN1 → volumen OP_IGUAL PORC | volumen OP_COMP PORC

VOLUMEN2 → volumen OP_ASSIGN PORC

MUTE1 → mute OP_IGUAL BOOLEANO2

MUTE2 → mute OP_ASSIGN BOOLEANO2

MENSAJE1 → mensaje OP_IGUAL TEXTO

MENSAJE2 → mensaje OP_ASSIGN TEXTO

EMAIL1 → email_notif OP_IGUAL EMAIL

EMAIL2 → email_notif OP_ASSIGN EMAIL

ALARMA → alarma_.ESTADO1 | alarma_.ACTIVADA1

ACTIVADA1 → activada OP_IGUAL BOOLEANO2

ACTIVADA2 → activada OP_ASSIGN BOOLEANO2

OPERADORES

OP_IGUAL → == | !=

OP_COMP → > | < | >= | <=

OP_ASSIGN → =

BOOLEANO

BOOLEANO1 → true | false

BOOLEANO2 → on | off

TEXTO → "string"

HORA

HORA → time

FECHA

FECHA → date

EMAIL

EMAIL → email@gmail.com

TEMP → NUM°C

LUX → NUMlux

PORC → NUM%

Análisis Léxico

El analizador léxico constituye la primera fase de procesamiento de nuestro trabajo para el lenguaje SMART-HOME. Su función es transformar el código fuente en una secuencia ordenada de componentes léxicos o tokens válidos, descartando los elementos redundantes para el análisis sintáctico (espacios en blanco y comentarios) y aislando las irregularidades del texto. El núcleo principal se encuentra estructurado en la clase **SmartHomeLexer**.

Definición de Componente Léxicos.

Para separar las cadenas literales se definió la clase **TT** (Token Types), la cual centraliza todos los tipos de componentes reconocidos por lexer mediante constantes. Los componentes léxicos se clasifican en las siguientes categorías:

- ❖ Palabras reservadas y control de flujo: **WHEN, IF, THEN, ELSE, DO, END, EVERY**.
- ❖ Operadores lógicos y constantes booleanas: encargados de la evaluación formal de restricciones y de estados binarios para dispositivos. **AND, OR, NOT, TRUE, FALSE, ON, OFF**.
- ❖ Atributos Tipados Especializados (ATTR_*): identificadores que mapean directamente las propiedades físicas controlables de las entidades domóticas. **estado, brillo, temp_obj, posicion, volumen, email_notif, color, modo, temp_act, hora, fecha, mute, mensaje, activada**.
- ❖ Tokens especiales de control: marcadores internos de comportamientos. **COMENTARIO, DESCONOCIDO, EOF**.
- ❖ Modos y Atributos Cromáticos Discretos: cadenas de control para sistemas de climatización (**FRIO, CALOR, VENT**) e iluminación ambiental (**BLANCO, ROJO, AZUL**).
- ❖ Identificadores de Dispositivos Domóticos: clasificaciones genéricas que diferencian dinámicamente si una entidad opera como captadora de datos o ejecutora de acciones (**SENSOR, ACTUADOR**).
- ❖ Literales Cuantitativos con Unidad Física: representaciones de valores numéricos acoplados a magnitudes reales (**TEMPERATURA, PORCENTAJE, ILUMINANCIA, TIEMPO**).
- ❖ Literales Simples e Identificadores: estructuras de datos convencionales y cadenas de formato estricto (**HORA, FECHA, EMAIL, CADENA, ENTERO, FLOTANTE, IDENTIFICADOR**).
- ❖ Operadores y Símbolos de Puntuación: operadores de comparación relacional (**OP_EQ, OP_NEQ, OP_GTE, OP_LTE, OP_GT, OP_LT**), el operador

de asignación imperativa (**OP_ASIG**) y el delimitador de acceso de miembros (**PUNTO**).

Estructura de datos del token

Cada componente léxico identificado es instanciado como un objeto de la clase (**dataclass**) **Token**. Esta estructura preserva los atributos del código fuente original, facilitando la trazabilidad de errores durante etapas sintácticas:

- ❖ **tipo**: almacena la constante correspondiente de la clase **TT**.
- ❖ **valor**: almacena el lexema exacto que dió origen a ese token
- ❖ **linea**: registra el número de línea física donde reside el componente.
- ❖ **col**: guarda la columna exacta del carácter inicial del lexema dentro de dicha línea.

Mecanismo de reconocimiento y orden de precedencia

El método **_procesar_linea** recorre secuencialmente la cadena de entrada mediante un puntero (**pos**). En cada iteración, ignorando los caracteres de espacio en blanco mediante **_es_blanco**, la discriminación del texto se realiza mediante **_siguiente_token**.

La resolución de ambigüedades léxicas se garantiza de forma determinista mediante un orden estricto de precedencia dentro de **_siguiente_token**. La jerarquía es la siguiente:

1. Comentarios de línea (//): se interceptan al inicio para ignorar el resto de la estructura lineal.
2. Cadenas de texto ("..."): delimitadas estrictamente por comillas dobles sin permitir saltos de línea intermedios.
3. Direcciones de correo electrónico: evaluadas tempranamente si el carácter inicial es un componente válido y se detecta la existencia del símbolo @ adelante en el flujo.
4. Magnitudes físicas: reconocimiento ordenado de TEMPERATURA (que admite prefijo negativo), seguido de las magnitudes estrictamente positivas ILUMINANCIA (lux), PORCENTAJE (%) y TIEMPO (s, m, h). El lexer comprueba activamente que no existan caracteres alfabéticos inmediatamente posteriores para evitar falsos positivos (por ejemplo, impidiendo que la palabra "5min" sea fragmentada erróneamente como un tiempo "5m" seguido del identificador "in").
5. Estructuras de tiempo y calendario: formatos estructurados fijos de FECHA (DD/MM/AAAA) y HORA (HH:MM), validando matemáticamente que las horas no excedan 23, los minutos 59, y los componentes de fecha se ubiquen en rangos reales.

6. Operadores compuestos y simples: evaluación de operadores de dos caracteres (==, !=, >=, <=) antes de pasar a los operadores unitarios (>, <, =, .), evitando lecturas parciales defectuosas.
7. Valores numéricos puros: clasificación de literales en **FLOTANTE** si contienen un punto decimal embebido, o **ENTERO** en caso contrario.
8. Identificadores y palabras clave: captura de bloques alfanuméricos que inician con letras o guión bajo, derivando su resolución mediante **_clasificar**.

Resolución de contexto

Para evitar la sobrecarga del analizador sintáctico, el lexer cuenta con una memoria contextual interna (self._ultimo_tipo, self._ctx_prefijo, self._ctx_attr_tipo). El método **_clasificar** utiliza esta memoria de la siguiente forma:

- ❖ Si una palabra coincide con la estructura de un atributo (como "estado" o "brillo"), sólo se le asignará su token de atributo específico (**TT.ATTR_ESTADO**, **TT.ATTR_BRILLO**) si el token inmediatamente anterior guardado en el historial corresponde a un **TT.PUNTO**. De no cumplirse esta precondition, la palabra se degrada a un **TT.IDENTIFICADOR** genérico.
- ❖ Los prefijos de dispositivos mapeados en las tuplas **PREFIJOS_SENSOR** y **PREFIJOS_ACTUADOR** se encuentran ordenados de mayor a menor longitud. Esto asegura que un identificador específico (como "sensor_humedad") no sea asimilado erróneamente por un prefijo de menor longitud (como "sensor_humo").

Gestión de errores

Protocolo de recuperación ante caracteres inválidos

Si el flujo de caracteres se sitúa en un elemento que no genera ninguna coincidencia con las reglas de bajo nivel de **_siguiente_token**, el método principal activa su rutina de contingencia:

1. Inserta una descripción detallada en la lista **self.errores**, documentando la línea y columna del carácter no reconocido.
2. Genera y acopla a la secuencia un token de tipo **TT.DESCONOCIDO** que encapsula dicho carácter.
3. Fuerza un incremento unitario del puntero (pos += 1). Este desplazamiento manual rompe cualquier posibilidad de bucles infinitos, permitiendo al lexer continuar analizando el resto del documento para detectar y reportar múltiples anomalías léxicas en una única pasada.

Sistema de advertencias semánticas tempranas

Durante el proceso de tokenización, cada vez que el autómata detecta un literal numérico físico (TEMPERATURA, PORCENTAJE, ILUMINANCIA), invoca al método de validación **_validar_rango**. A través del rastreo contextual del dispositivo actual y el atributo seleccionado, la función realiza una consulta directa sobre la estructura **TABLA_RANGOS**.

Si el valor numérico del lexema —procesado dinámicamente por la rutina auxiliar correspondiente— viola los límites físicos parametrizados (por ejemplo, definir un valor fuera de la cota de -10.0°C a 50.0°C para un sensor_temp), el sistema registra una advertencia explícita en self.advertencias. Al tratarse de una advertencia semántica temprana, el proceso léxico no se interrumpe, permitiendo la generación completa del flujo de tokens.

MÉTODO DE FUNCIONAMIENTO

El funcionamiento del programa comienza con la apertura del selector de archivos. En esta instancia, el usuario debe seleccionar un archivo con extensión **.SMART** para cargar su contenido.

Si el usuario decide cerrar el selector sin seleccionar ningún archivo, el sistema permitirá ingresar líneas de código de manera manual a través de la interfaz.

Análisis Sintáctico

El analizador sintáctico construye la segunda fase de procesamiento de nuestro compilador. Su función principal es validar que la secuencia de componentes léxicos, producida en la etapa anterior, cumpla con las reglas gramaticales definidas por la gramática, construyendo simultáneamente la representación intermedia y final del programa. Su núcleo estructural se encuentra implementado mediante **PLY** (Python Lex-Yacc).

Estructura del Adaptador Léxico

Debido a que el análisis léxico se implementa de manera manual mediante **SmarthHome**, se diseñó un patrón adaptador para acoplar dicho lexer con los requerimientos de **PLY**.

- Filtrado de Tokens Irrelevantes: La clase **LexerAdaptador** recibe la lista completa de tokens del código fuente y descarta de manera temprana aquellos componentes que carecen de valor estructural para la sintaxis, tales

como comentarios (COMENTARIO), espacios vacíos, tokens desconocidos (DESCONOCIDO) y marcadores de fin de archivo (EOF).

- Envoltura Protocolar (**_PLYToken**): PLY invoca internamente al método .token() y espera recibir un objeto provisto de propiedades específicas. El adaptador encapsula cada instancia de la estructura Token del lexer dentro de un objeto **_PLYToken**, realizando el siguiente mapeo paramétrico:
 - .type: recibe el string de tipo léxico (tok.tipo), el cual es utilizado directamente por las producciones gramaticales p_* .
 - .value: preserva la instancia completa de Token generada en la fase léxica. Esto permite que, dentro de cada regla gramatical, las expresiones $p[i]$ expongan los metadatos completos de la estructura, incluyendo .linea, .col y .valor.
 - .lineno y .lexpos: exponen la línea y columna correspondientes para el seguimiento de la posición.
- Compatibilidad Dinámica de Error: La clase **_PLYToken** se declara deliberadamente sin atributos restringidos. Esta decisión arquitectónica evita excepciones internas cuando el motor LALR(1) de PLY intenta inyectar atributos dinámicos (como .lexer) durante sus rutinas internas de recuperación ante errores sintácticos.

Resolución de Ambigüedades y Precedencia

Para evaluar de forma unívoca las expresiones booleanas compuestas y garantizar una jerarquía coherente en las condiciones, el parser establece explícitamente una tabla de precedencia y asociatividad de operadores (precedence):

1. Operador OR (OR): Asociatividad hacia la izquierda, con el nivel de precedencia más bajo.
2. Operador AND (AND): Asociatividad hacia la izquierda, con un nivel de precedencia intermedio.
3. Operador Unario NOT (NOT): Asociatividad hacia la derecha, con el nivel de precedencia más alto.

Diseño Gramatical y Traducción Dirigida por Sintaxis

El parser no construye un Árbol de Sintaxis Abstracta (AST) explícito en memoria; en su lugar, implementa un esquema de Traducción Dirigida por Sintaxis (SDT) orientado a la generación directa de una interfaz web (Dashboard HTML).

Estrategia de Producciones Divididas (Cabeceras y Cuerpos)

Para modelar bloques condicionales y temporizadores sin perder el orden visual de los elementos generados, las producciones principales de control (WHEN, EVERY, IF) se dividen en dos fases sintácticas: apertura y cierre.

- Apertura de Bloque (`bloque_when_open`, `bloque_every_open`, `if_open`): Al emparejar los componentes declarativos iniciales (por ejemplo, `WHEN` `condicion DO`), la regla gramatical emite de forma inmediata la cabecera HTML correspondiente con su estilo de sección (`CSS_SECCION`) y renderiza los bloques de los sensores involucrados. Adicionalmente, invoca al gestor de estado para empujar un nuevo registro en la pila de acumuladores (`push_acum()`).
- Cierre de Bloque (`bloque_when`, `bloque_every`, `condicional`): Tras evaluar de forma recursiva todas las instrucciones del no-terminal cuerpo, el parser procede a volcar (`volcar()`) en la salida HTML todos los actuadores acumulados, emitiendo cajas visuales agrupadas (`CSS_ACTUADOR`) antes de cerrar las etiquetas de sección y desapilar el contexto (`pop_acum()`).

Manejo de Contextos Anidados

La gramática distingue semánticamente entre condicionales de nivel raíz (`condicional`) y condicionales internos (`condicional_inner`), los cuales pueden declararse dentro del cuerpo de otras sentencias. Al abrir un bloque condicional anidado (`if_inner_open`), el parser vacía previamente los actuadores pendientes del nivel superior, garantizando una jerarquía visual limpia y respetando el ámbito (scope) físico de cada dispositivo.

Análisis Semántico y Control de Especificaciones

Las validaciones de significado y coherencia física del lenguaje ocurren de manera incrustada durante las reducciones gramaticales, delegadas en la clase de control `global_Estado`.

Validación de Atributos de Actuadores (`chk_attr`)

Cada vez que se reduce una asignación o se consulta un atributo de un actuador, el sistema consulta el diccionario estricto de especificaciones `ESPECIFICACION_ATRIBUTOS`. Esta estructura mapea la tupla (`TipoActuador`, `TipoAtributo`) contra sus reglas de uso:

- Control de Permisos (Solo Lectura): Identifica y bloquea intentos de asignación imperativa sobre atributos que representan medidas puramente informativas. Por ejemplo, intentar asignar un valor al atributo `temp_act` de un aire acondicionado o a la hora del reloj generará un error semántico preciso informando que la propiedad es de "Solo Lectura".
- Compatibilidad de Tipos de Datos: Verifica que el token asignado pertenezca al conjunto del dominio permitido. Si se asigna un literal numérico incompatible o una constante fuera del dominio de un dispositivo (por

ejemplo, asignar un valor que no sea PORC al atributo brillo de un foco), se emitirá una anomalía detallada con número de línea y columna.

Validación de Sensores (chk_sensor)

En las reducciones condicionales (expr_sensor), el método chk_sensor coteja el valor evaluado contra la tabla TIPOS_VALOR_SENSOR. Se valida estrictamente que un sensor_temp sea comparado con una TEMPERATURA, un sensor_humedad con un PORCENTAJE, o un sensor_movimiento con un booleano (TRUE / FALSE).

Gestión de Errores Sintácticos

El tratamiento de fallos en la estructura del código está gobernado por la rutina p_error(p).

- Fin de Archivo Inesperado: Si el parámetro p es None, el parser registra un error sintáctico indicando una ruptura inesperada antes de completar el árbol de derivación.
- Tokens Inesperados: Si se intercepta una violación gramatical, el manejador accede a p.value para recuperar el token original. Con ello, inserta un registro exacto en _st.errores_sint especificando la línea, columna, valor y tipo del lexema conflictivo que impidió la reducción.

Modos de Compilación y Ejecución

El flujo final del parser, orquestado a través de la función compilar, consolida los reportes generados por todas las capas:

1. Ejecución Léxica: Se genera el inventario total de tokens, recogiendo errores léxicos y advertencias semánticas de rango.
2. Ejecución Sintáctico-Semántica: Se inicializa la cabecera HTML y se procesa la estructura mediante _parser.parse().
3. Evaluación de Integridad: El compilador opera en modalidad transaccional. Si existe al menos un error léxico, sintáctico o semántico, el sistema cancela la producción final del archivo HTML y emite el listado estructurado de fallos por consola, asegurando que únicamente código 100% válido genere vistas funcionales del Dashboard domótico.

Apéndice

Vídeo explicativo sobre nuestro proyecto: <https://youtu.be/yQOo3rIoEI8>